

YOLOv1 Model Architecture

1 Background

2 Model Architecture

YOLOv1 frames object detection as a single end-to-end regression problem. Given a $448 \times 448 \times 3$ image, the model outputs a tensor of size $S \times S \times (B \cdot 5 + C)$, where $S = 7$, $B = 2$, and $C = 20$ for PASCAL VOC. Thus, each grid cell predicts two bounding boxes and twenty conditional class probabilities. This section analyses the model part-by-part and synthesizes how the components integrate into a unified dense prediction system.

Figures 1, 2, 3, and 4 illustrate the full YOLOv1 architecture.

2.1 High-Level Architecture

YOLOv1 consists of:

1. A deep convolutional backbone (24 conv layers).
2. A fully-connected detection head that produces the $7 \times 7 \times 30$ output tensor.

The overall structure is shown in Figure 1.

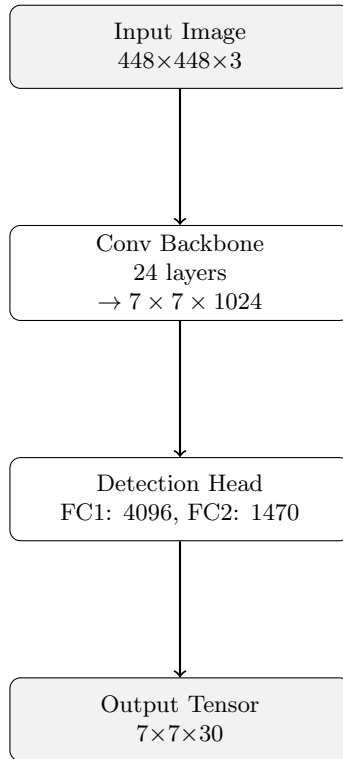


Figure 1: High-level YOLOv1 architecture.

2.2 Backbone Network

The backbone is inspired by GoogLeNet and Network-in-Network. It alternates 1×1 bottleneck convolutions with 3×3 convolutions, combined with max-pooling to reduce spatial resolution:

$$448 \rightarrow 224 \rightarrow 112 \rightarrow 56 \rightarrow 28 \rightarrow 14 \rightarrow 7.$$

A layer-by-layer schematic with spatial sizes is shown in Figure 2.

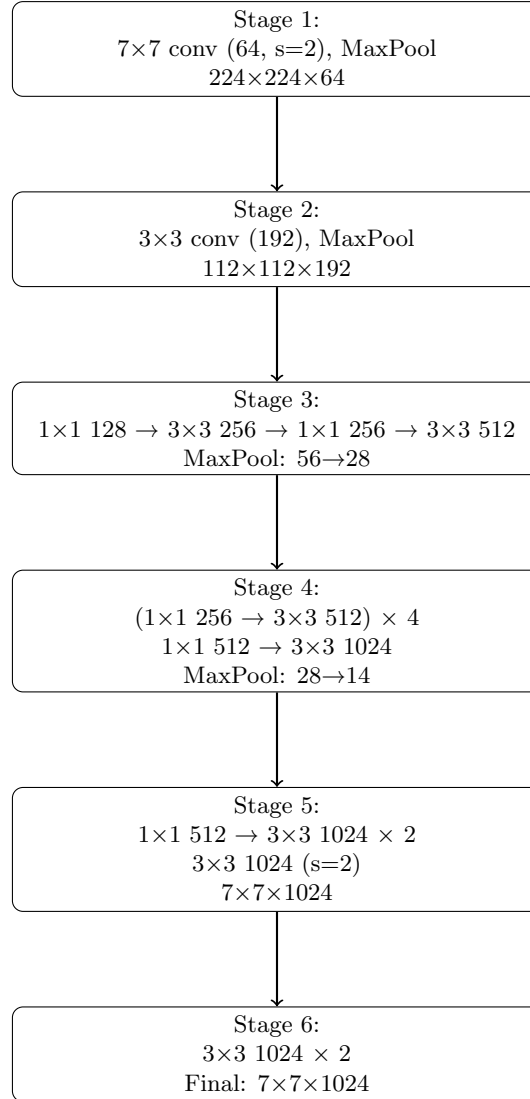


Figure 2: YOLOv1 backbone architecture with spatial resolution after each stage.

2.3 Grid-Based Prediction

The 7×7 final feature map corresponds to a 7×7 image grid. Objects are assigned to the cell containing their center. Each cell predicts:

- Two bounding boxes (x, y, w, h, c) ,
- Twenty class probabilities.

Figure 3 illustrates the grid-based formulation.

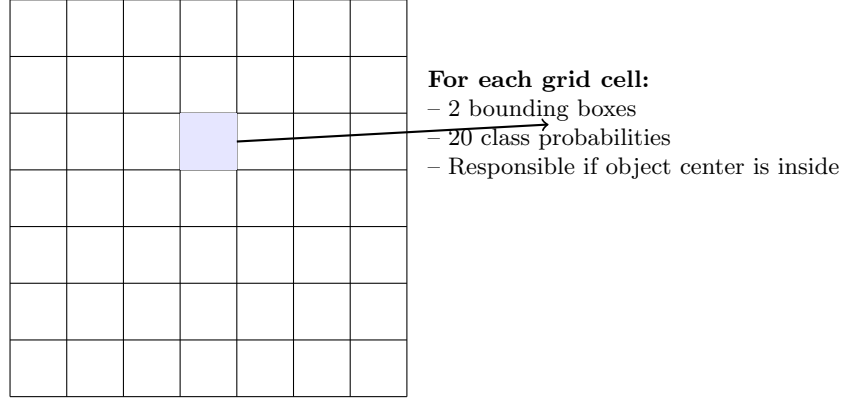


Figure 3: YOLOv1 grid-based prediction structure.

2.4 Output Tensor Structure

Each grid cell outputs a 30-dimensional vector:

$$[x_1, y_1, w_1, h_1, c_1, x_2, y_2, w_2, h_2, c_2, p_1, \dots, p_{20}].$$

This is illustrated in Figure 4.

Box 1: x_1, y_1, w_1, h_1, c_1
Box 2: x_2, y_2, w_2, h_2, c_2
Class Probabilities: $p_1, \dots, p_{20}$

Figure 4: Output vector for one YOLOv1 grid cell (30D).

2.5 Synthesis

Combining the backbone and detection head yields a unified dense detector:

1. The backbone produces a structured 7×7 grid of deep features.
2. The detection head performs direct regression to bounding boxes and class scores.
3. The model eliminates region proposals entirely, enabling real-time performance.

2.6 Code sketch: YOLOv1-style architecture in PyTorch

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class ConvBlock(nn.Module):
    """Conv2d -> BatchNorm (optional) -> LeakyReLU."""
    def __init__(self, in_ch, out_ch, k, s=1, p=0):
        super().__init__()
```

```

        self.conv = nn.Conv2d(in_ch, out_ch, kernel_size=k, stride=s, padding=p, bias=False)
        self.bn = nn.BatchNorm2d(out_ch)
        self.act = nn.LeakyReLU(0.1, inplace=True)

    def forward(self, x):
        return self.act(self.bn(self.conv(x)))

class YoloV1Backbone(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            # Stage 1
            ConvBlock(3, 64, 7, s=2, p=3),
            nn.MaxPool2d(2, 2),

            # Stage 2
            ConvBlock(64, 192, 3, s=1, p=1),
            nn.MaxPool2d(2, 2),

            # Stage 3
            ConvBlock(192, 128, 1),
            ConvBlock(128, 256, 3, p=1),
            ConvBlock(256, 256, 1),
            ConvBlock(256, 512, 3, p=1),
            nn.MaxPool2d(2, 2),

            # Stage 4 (×4 block)
            ConvBlock(512, 256, 1),
            ConvBlock(256, 512, 3, p=1),
            ConvBlock(512, 256, 1),
            ConvBlock(256, 512, 3, p=1),
            ConvBlock(512, 256, 1),
            ConvBlock(256, 512, 3, p=1),
            ConvBlock(512, 256, 1),
            ConvBlock(256, 512, 3, p=1),
            ConvBlock(512, 512, 1),
            ConvBlock(512, 1024, 3, p=1),
            nn.MaxPool2d(2, 2),

            # Stage 5
            ConvBlock(1024, 512, 1),
            ConvBlock(512, 1024, 3, p=1),
            ConvBlock(1024, 512, 1),
            ConvBlock(512, 1024, 3, p=1),
            ConvBlock(1024, 1024, 3, p=1),
            ConvBlock(1024, 1024, 3, s=2, p=1),

            # Stage 6
            ConvBlock(1024, 1024, 3, p=1),
            ConvBlock(1024, 1024, 3, p=1),
        )

    def forward(self, x):
        return self.features(x) # (B, 1024, 7, 7) for 448x448 input

```

```

class YoloV1(nn.Module):
    def __init__(self, S=7, B=2, C=20):
        super().__init__()
        self.S, self.B, self.C = S, B, C
        self.backbone = YoloV1Backbone()
        self.fc1 = nn.Linear(1024 * S * S, 4096)
        self.dropout = nn.Dropout(0.5)
        self.fc2 = nn.Linear(4096, S * S * (B * 5 + C))

    def forward(self, x):
        B = x.size(0)
        x = self.backbone(x)          # (B, 1024, 7, 7)
        x = x.view(B, -1)             # flatten
        x = F.leaky_relu(self.fc1(x), 0.1)
        x = self.dropout(x)
        x = self.fc2(x)                # (B, 7*7*(B*5+C))
        x = x.view(B, self.S, self.S, self.B * 5 + self.C)
        return x

```

3 Model Training

Training YOLOv1 involves (1) ImageNet pretraining for the backbone, (2) conversion to a detection network, (3) optimizing the multi-part YOLO loss, and (4) using a carefully scheduled learning rate and regularization pipeline. This section explains each training component and provides visualizations (Figure 5) and reference code snippets.

3.1 Pretraining on ImageNet

The first 20 convolutional layers of YOLOv1 are pretrained as an ImageNet classifier. A global average pooling layer and a fully-connected softmax classifier are appended to perform 1000-way classification. This stage enables the model to learn low- and mid-level features such as edges, textures, and object parts.

The pretraining strategy:

- Input resolution: 224×224 .
- Optimizer: SGD, batch size 128.
- Achieves $\sim 88\%$ top-5 accuracy.

After pretraining, the classification head is removed, and the remaining convolutional backbone is transferred to the detection network.

3.2 Converting to a Detection Network

After loading pretrained weights, YOLOv1 appends:

- Four additional convolution layers.
- Two fully-connected layers:

$$7 \times 7 \times 1024 \rightarrow 4096 \rightarrow 7 \times 7 \times 30.$$

The newly added layers are randomly initialized, whereas all backbone layers are initialized from ImageNet pretraining.

3.3 YOLOv1 Loss Function

YOLOv1 uses a multi-part sum-squared-error loss consisting of:

1. **Localization loss** for coordinates (x, y, w, h) .
2. **Confidence loss** for predicted IoU.
3. **Classification loss** for C conditional class probabilities.

The full YOLOv1 loss (for $S \times S$ grid and B boxes per cell) is:

$$\begin{aligned}
L = & \lambda_{\text{coord}} \sum_{i=1}^{S^2} \sum_{j=1}^B \mathbb{K}_{ij}^{\text{obj}} [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2] \\
& + \lambda_{\text{coord}} \sum_{i=1}^{S^2} \sum_{j=1}^B \mathbb{K}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \\
& + \sum_{i=1}^{S^2} \sum_{j=1}^B \mathbb{K}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\
& + \lambda_{\text{noobj}} \sum_{i=1}^{S^2} \sum_{j=1}^B \mathbb{K}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\
& + \sum_{i=1}^{S^2} \mathbb{K}_i^{\text{obj}} \sum_{c=1}^C (p_i(c) - \hat{p}_i(c))^2.
\end{aligned}$$

Hyperparameters:

$$\lambda_{\text{coord}} = 5, \quad \lambda_{\text{noobj}} = 0.5.$$

These weights ensure:

- Localization receives high importance.
- Background cells do not dominate training.

3.4 Learning Rate Schedule and Optimization

YOLOv1 uses SGD with momentum:

- Optimizer: SGD, momentum = 0.9
- Weight decay: 5×10^{-4}
- Batch size: 64

The learning rate schedule:

1. Warmup: $10^{-3} \rightarrow 10^{-2}$ over first few epochs.
2. Train at 10^{-2} for 75 epochs.
3. Reduce to 10^{-3} for 30 epochs.
4. Reduce to 10^{-4} for final 30 epochs.

This schedule stabilizes high-variance gradients in early training and allows late-stage fine-tuning.

3.5 Regularization and Data Augmentation

YOLO uses:

- Dropout ($p = 0.5$) after the first fully connected layer.
- Data augmentation:
 - Random scaling and translations ($\pm 20\%$).
 - Exposure and saturation jitter in HSV space (factor up to 1.5).

These augmentations reduce overfitting, especially in small datasets like VOC.

3.6 Training Loop Overview

The training loop structure is visualized in Figure 5.

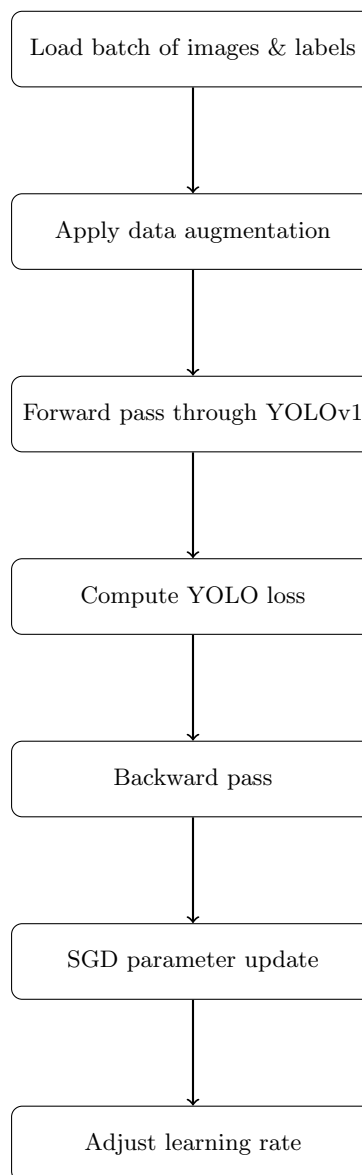


Figure 5: Training loop for YOLOv1.

3.7 Training Loop Code Example

Below is a simplified PyTorch-style training loop for YOLOv1. Code is shown using `verbatim` as required.

```
for epoch in range(num_epochs):
    model.train()
    adjust_learning_rate(optimizer, epoch)

    for images, targets in train_loader:
        images = images.to(device)
        targets = targets.to(device)

        optimizer.zero_grad()
        preds = model(images)

        loss = yolo_loss(preds, targets)
        loss.backward()

        optimizer.step()

    # (optional) evaluate mAP or validation loss here
```

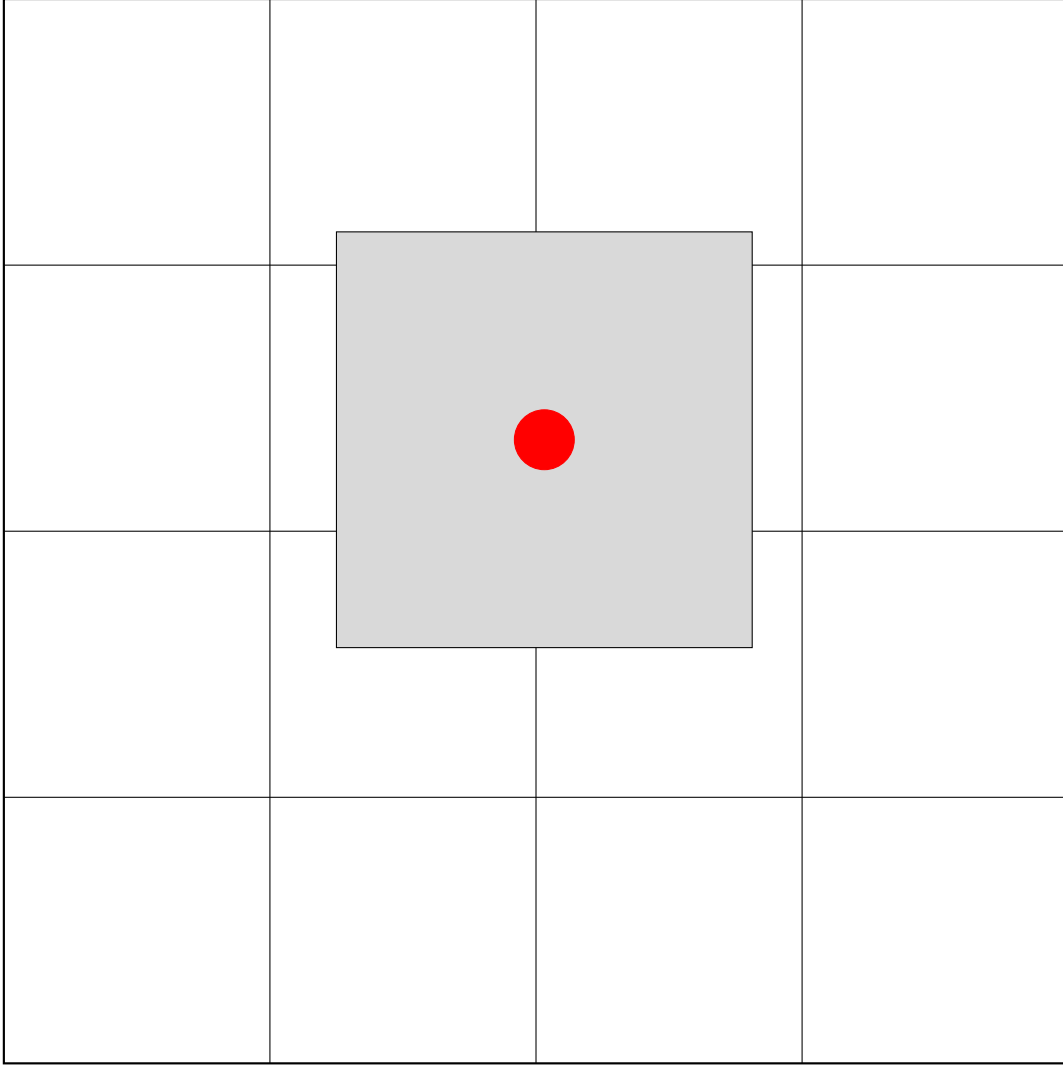
4 Minimal CPU-Ready Working Example (MWE-CPU)

The full YOLOv1 model is computationally heavy and relies on a large-scale dataset. To examine the architecture in a controlled setting and to make its dynamics visible without GPU resources, a minimal CPU-ready working example is constructed. The objective is not to approximate YOLOv1 performance, but to isolate its structural ideas—grid-based reasoning, joint regression, and dense predictions—within a tractable environment. The resulting setup is intentionally compact, yet retains the core mechanisms that define the model’s behavior.

4.1 Synthetic Dataset

The synthetic dataset abstracts away natural-image complexity by generating 64×64 black images containing a single white rectangle. The rectangle’s size and location vary across samples, and its center determines the responsible grid cell in a 4×4 partition. This controlled construction permits direct observation of how the model distributes responsibility across cells and how it internalizes spatial cues.

Figure 6 illustrates a typical instance.



Example image with rectangle and 4×4 grid

Figure 6: Synthetic data example. A single rectangle provides a controlled signal for localization.

The dataset generation code is included for clarity regarding target structure:

```
class SyntheticSquareDataset(Dataset):
    def __init__(self, num_samples=2000, image_size=64, S=4):
        self.num_samples = num_samples
        self.image_size = image_size
        self.S = S
        self.cell = image_size // S

    def __getitem__(self, idx):
        img = torch.zeros(3, 64, 64)
        w = random.randint(8, 32)
        h = random.randint(8, 32)
```

```

x1 = random.randint(0, 64 - w - 1)
y1 = random.randint(0, 64 - h - 1)
x2, y2 = x1 + w, y1 + h
img[:, y1:y2, x1:x2] = 1.0

cx = (x1 + x2) / 2
cy = (y1 + y2) / 2
gx = int(cx // self.cell)
gy = int(cy // self.cell)

target = torch.zeros(self.S, self.S, 6)
tx = (cx - gx*self.cell) / self.cell
ty = (cy - gy*self.cell) / self.cell
target[gy, gx] = torch.tensor([tx, ty, w/64, h/64, 1.0, 1.0])
return img, target

```

4.2 Tiny YOLO Architecture

To preserve YOLO’s conceptual design while remaining viable on a CPU, a lightweight network is used. It mirrors the original architecture’s progression from spatial reduction to dense prediction, but scales all dimensions to a level appropriate for rapid experimentation. Figure 7 summarizes the structure.

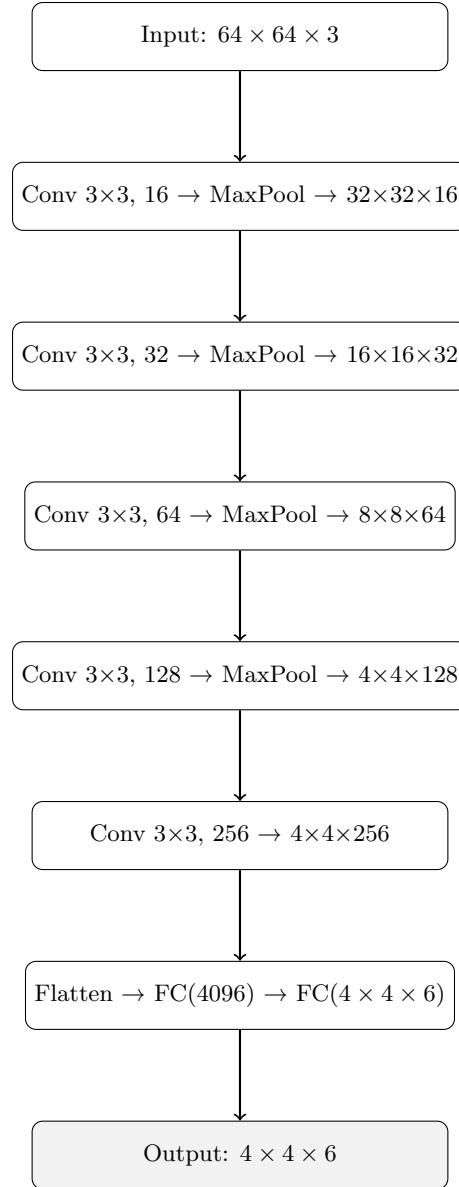


Figure 7: Structure of the tiny YOLO model. The architecture retains the key transition from convolutional encoding to dense prediction.

Implementation:

```

class TinyYolo(nn.Module):
    def __init__(self, S=4):
        super().__init__()
        self.S = S
        self.features = nn.Sequential(
            nn.Conv2d(3,16,3,padding=1), nn.LeakyReLU(0.1), nn.MaxPool2d(2),
            nn.Conv2d(16,32,3,padding=1), nn.LeakyReLU(0.1), nn.MaxPool2d(2),
            nn.Conv2d(32,64,3,padding=1), nn.LeakyReLU(0.1), nn.MaxPool2d(2),
            nn.Conv2d(64,128,3,padding=1), nn.LeakyReLU(0.1), nn.MaxPool2d(2),
            nn.Conv2d(128,256,3,padding=1), nn.LeakyReLU(0.1)
        )

```

```

self.fc1 = nn.Linear(256*4*4, 4096)
self.fc2 = nn.Linear(4096, S*S*6)

def forward(self, x):
    B = x.size(0)
    x = self.features(x)
    x = x.view(B, -1)
    x = torch.leaky_relu(self.fc1(x), 0.1)
    x = self.fc2(x)
    return x.view(B, self.S, self.S, 6)

```

4.3 Loss Formulation

The loss follows the structure of YOLOv1, though scaled to the simplified setting. Given the absence of multiple objects and classes, the formulation highlights how the model negotiates coordinate precision and confidence calibration. The loss is:

$$L = \lambda_{\text{coord}} \| (x, y, w^{1/2}, h^{1/2}) - (\hat{x}, \hat{y}, \hat{w}^{1/2}, \hat{h}^{1/2}) \|^2 + (C - \hat{C})_{\text{obj}}^2 + \lambda_{\text{noobj}} (C - \hat{C})_{\text{noobj}}^2 + (p - \hat{p})_{\text{obj}}^2.$$

Code:

```

def tiny_yolo_loss(pred, target, lambda_coord=5.0, lambda_noobj=0.5):
    obj = target[...,4] > 0.5
    noobj = ~obj

    xy_loss = ((pred[...,0:2][obj] - target[...,0:2][obj])**2).sum()
    pred_wh = torch.sqrt(torch.clamp(pred[...,2:4], min=1e-6))
    targ_wh = torch.sqrt(torch.clamp(target[...,2:4], min=1e-6))
    wh_loss = ((pred_wh[obj] - targ_wh[obj])**2).sum()

    conf_obj = ((pred[...,4][obj] - target[...,4][obj])**2).sum()
    conf_no = ((pred[...,4][noobj] - target[...,4][noobj])**2).sum()
    cls_loss = ((pred[...,5][obj] - target[...,5][obj])**2).sum()

    return lambda_coord*(xy_loss + wh_loss) + \
        conf_obj + lambda_noobj*conf_no + cls_loss

```

4.4 Training Procedure

The training process, summarized in Figure 8, follows a standard forward–backward–update routine. While simple, it makes explicit how the model accumulates evidence across epochs, steadily refining its estimate of spatial responsibility and bounding geometry.

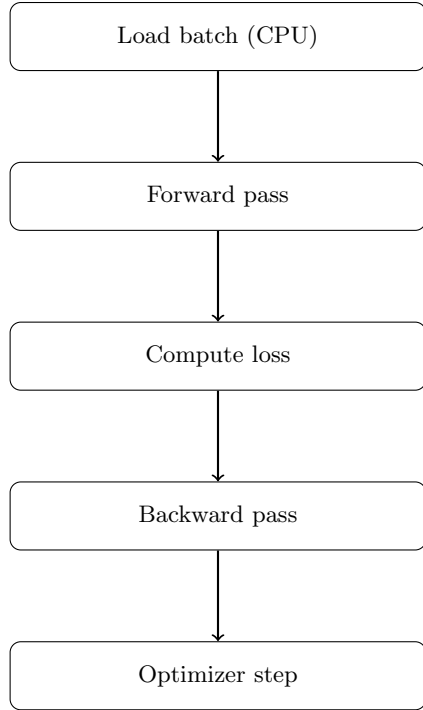


Figure 8: Training flow for the tiny YOLO model.

Training code:

```

device = torch.device("cpu")
model = TinyYolo(S=4).to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(10):
    for imgs, targets in train_loader:
        imgs, targets = imgs.to(device), targets.to(device)
        optimizer.zero_grad()
        preds = model(imgs)
        loss = tiny_yolo_loss(preds, targets)
        loss.backward()
        optimizer.step()
    print(f"Epoch {epoch}: loss={loss.item():.4f}")

```

4.5 Behavioral Observations

Although the model is intentionally constrained, its learning dynamics exhibit several patterns characteristic of YOLO-style detectors. Cell responsibility emerges early: confidence values begin to polarize, and the network identifies the appropriate grid cell long before it refines the bounding box geometry. As training progresses, width and height predictions stabilize, and the model converges to a coherent mapping between spatial layout and output encoding. The system, despite its scale, captures the causal structure of the detection task.